

# 1 Code overview

`models.py`

Implements the five architectures required in Q 3: **Canonisation\_Net**, **Symmetrization\_Net**, **Sampled\_Symmetrization\_Net**, **Linear\_eq\_Net** (built from two equivariant layers), and **AugmentedInvariantNet**. Each class exposes `forward(X)` with  $X \in \mathbb{R}^{N \times d}$  drawn from a synthetic white-Gaussian *matrix* generated by `torch.randn`.

`utils.py` • Permutation helpers: `create_permutations_sampled`.

- *Unit tests* for invariance/equivariance (`test_canonization_net, ...`).
- Training routine `train_variance_net` that learns variance purely from permutation augmentations.

`main.py`

Orchestrates the workflow:

- (a) runs each invariance test 50 times and prints the failure rate;
- (b) trains the variance model for 200 epochs on a fixed set of size  $N = 50$ ,  $d = 5$ ;
- (c) re-tests the trained model for invariance.

Dependencies are limited to `torch` and `numpy`; the script detects a GPU via `torch.cuda.is_available()` and moves tensors accordingly.

## 2 Permutation–invariance sanity check

### test Permutation invariance

For every network we apply 50 random permutations  $P \in S_N$  to a fixed set  $X$  and record a *failure* when the  $\ell_\infty$ –distance between the original and permuted outputs exceeds a model-specific tolerance:

- (a,b,d) fail if  $\|f(X) - f(PX)\|_\infty > 10^{-5}$ ;
- (c) fail if  $\|f(X) - f(PX)\|_\infty > 2 \times 10^{-1}$   
(higher threshold to reflect the Monte-Carlo approximation of  $S_N$ );
- (e) fail if  $\|f(X) - f(PX)\|_\infty > 5 \times 10^{-2}$   
(relaxed tolerance to probe the invariance learned via augmentation).

Finally, for each model we report the *failure rate*

$$\text{Failure rate} = \frac{\# \text{ failed permutations}}{\# \text{ permutations tested}},$$

i.e. the fraction of the 50 sampled permutations that violate their respective tolerance.

| Network                              | % non-invariant ↓ | Pass? |
|--------------------------------------|-------------------|-------|
| Canonisation                         | 0.00              | ✓     |
| Full symmetrisation                  | 0.00              | ✓     |
| Sampled symmetrisation ( $K = 256$ ) | 0.94              | ✗     |
| $S_N$ -equivariant linear layers     | 0.00              | ✓     |
| Learned-invariant (augmentations)    | 0.28              | ✓*    |

Table 1: Failure rate for each network.

## Discussion

- **Exact symmetry methods.** Canonisation, full symmetrisation, and linear  $S_N$ -equivariant layers are provably invariant; failures are zero within numerical precision.
- **Monte-Carlo method (c).** Sampled symmetrisation averages over  $K = 0.05 \cdot 8!$  permutations—  $\approx 0.05 |S_N|$  for  $N = 8$  - so estimator variance might explain the  $\approx 1\%$  failure rate. Increasing  $K$  should decrease the error.
- **Learned invariance (e).** The unconstrained MLP acquires symmetry through augmentation; after  $\sim 250$  epochs with  $\sim 500$  augmentations per epoch the failure rate falls below 0.3%.

\* Both stochastic models (c and e) pass with a looser tolerance, We intentionally kept a looser threshold here to reveal the *trend* toward invariance and to assess given our time and computation resources how far each model has progressed toward full invariance.